

Generative Adversarial Networks Based Low-Rate Denial of Service Attack Detection and Mitigation in Software-Defined Networks

Manjuluri Anil Kumar[✉], Balaprakasa Rao Killi[✉], *Senior Member, IEEE*, and Eiji Oki[✉], *Fellow, IEEE*

Abstract—Low-rate Denial of Service (LDoS) attacks use short, regular bursts of traffic to exploit vulnerabilities in network protocols. They are a major threat to network security, especially in Software-Defined Networking (SDN) frameworks. These attacks are challenging to detect and mitigate because of their low traffic volume, making it impossible to distinguish them from normal traffic. We propose a real-time LDoS attack detection and mitigation framework that can protect SDN. The framework incorporates a detection module that uses a deep learning model, such as a Generative Adversarial Network (GAN), to identify the attack. An efficient mitigation module follows detection, employing mechanisms to identify and filter harmful flows in real time. Deploying the framework into SDN controllers guarantees compliance with OpenFlow standards, thereby avoiding the necessity for additional hardware. Experimental results demonstrate that the proposed system achieves a detection accuracy of over 99.98% with an average response time of 8.58 s, significantly outperforming traditional LDoS detection approaches. This study presents a scalable, real-time methodology to enhance SDN resilience against LDoS attacks.

Index Terms—LDoS, SDN, GAN, attack detection and mitigation, OpenFlow.

I. INTRODUCTION

THE CONVENTIONAL control and data plane architecture of devices imposes constraints on the adaptability of network infrastructure, thereby restricting its capacity for innovation and expansion. Software-Defined Networking (SDN) is a networking method that separates the control plane, which determines the path of network traffic, from the requesting entity and the forwarding of data inside the network structure. The separation of the control plane from the data plane in SDN enables programmability, thereby simplifying the use of software for network configuration and control. Because of its centralized control methodology, SDN presents various security vulnerabilities. The centralized controller in a network is susceptible to significant attacks, as a single point of failure can cause the entire system to malfunction.

Received 17 February 2025; revised 15 August 2025; accepted 21 October 2025. Date of publication 24 October 2025; date of current version 29 December 2025. The associate editor coordinating the review of this article and approving it for publication was P. Papadimitriou. (*Corresponding author: Manjuluri Anil kumar.*)

Manjuluri Anil kumar and Balaprakasa Rao Killi are with the Department of Computer Science and Engineering, National Institute of Technology at Warangal, Warangal 506004, India (e-mail: ma23csr2r05@student.nitw.ac.in).

Eiji Oki is with the Graduate School of Informatics, Kyoto University, Kyoto 606-8501, Japan.

Digital Object Identifier 10.1109/TNSM.2025.3625278

Vulnerabilities in the southbound interface can result in the exploitation of protocols and man-in-the-middle attacks, which can compromise the controller. These vulnerabilities in the data layer place the entire network at risk. Further, policy conflicts can lead to vulnerabilities that allow for illegal or restricted traffic. Failure to effectively manage configuration and policies results in security vulnerabilities. Low visibility and monitoring can lead to delays in the identification of risks and increase their consequences. However, it also raises security issues, particularly in relation to Distributed Denial of Service (DDoS) attacks.

A DDoS attack is a type of Denial of Service (DoS) attack where attackers launch multiple attacks on a target, disrupting legitimate users' operations. Over the past two decades, DDoS attacks have evolved and undergone classification based on protocol vulnerabilities, resource focus, attack magnitude, and timing. They can use network-layer and transport-layer protocols, as well as application-layer protocols. DDoS attacks can be volumetric or non-volumetric, continuous, pulse-wave, periodic, or low-stealthy [1].

Low-rate Denial of Service (LDoS) attacks are a particular type of DoS attack. These LDoS attacks disrupt and degrade the service performance at the victim's end with minimal resource usage [2]. Three characteristics distinguish LDoS attacks from traditional DoS attacks. First, LDoS attacks target widespread adaptive mechanisms. These adaptive mechanisms effectively respond and adjust to LDoS attacks, exposing the target to a continuous attack without their knowledge. Second, LDoS attacks are highly concealed. The average volume of LDoS attack traffic is usually minimal due to the short burst duration. The resultant attack traffic closely resembles the brief bursts generated by benign applications. Finally, LDoS attacks are cost-effective. Unlike flooding attacks, LDoS attacks transmit minimal data and can be carried out by a single-source attacker [3]. LDoS attacks are harder to detect than DoS attacks because they are more discreet and can evade conventional defense mechanisms.

LDoS attacks target the hosts and critical services within the control plane to degrade the overall Quality of Service (QoS) in the SDN. Hosts in the SDN are vulnerable to LDoS attacks. LDoS attacks disrupt the normal Transmission Control Protocol (TCP) operation by sending intermittent bursts of high-speed data at a very low average rate. This keeps TCP in slow starts or fast retransmission stages. The intermittent bursts in regular network activity can efficiently conceal LDoS

attacks among background traffic. TCP-targeted LDoS attacks can also disrupt the southbound TCP interface between the control plane and the data plane, affecting the performance of the data plane. Detecting LDoS attacks within overall network traffic is challenging, particularly when the SDN controller has access to comprehensive traffic data. Consequently, an additional application is required to detect and mitigate LDoS attacks. The management of the entire network may be compromised when LDoS attacks target the SDN control plane. LDoS attacks have the potential to cause more damage than those in conventional networks if the control plane is unable to promptly identify and modify the network state. Current detection approaches for DoS attacks lack the ability to differentiate LDoS attack traffic from regular traffic. The identification and prevention of LDoS attacks continue to pose challenges due to their resemblance to authorized user access and flash event attacks.

Existing signature-based and behavior-based attack detection techniques rely on prior knowledge of attack patterns and network activity and require regular upgrades to protect against evolving attack methods [4], [5]. However, creating new signatures or updated profiles for these detectors requires significant time and resources, often requiring network administrative skills. To address these challenges, several advanced detection methods have been developed, including supervised, unsupervised, and mixed anomaly detection approaches [6]. Unsupervised learning-based anomaly detection methods assume a normal distribution of the data. Any data point that is significantly different from the expected pattern is considered an anomaly [7]. Supervised anomaly detection involves training a classifier using a dataset classified as either “anomaly” or “normal” [8]. However, this requires thorough training of both “anomaly” and “normal” observations, making it difficult for systems to accurately detect and recognize these patterns.

Deep learning methods like Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) are utilized to achieve favorable outcomes in anomaly detection tasks [9]. Nevertheless, these methods face difficulties in efficiently working in network anomaly detection situations because of the large number of normal samples and shortage of anomalous samples. The data imbalance and lack of previous information pose significant challenges in the detection process, especially for zero-day attacks. The rapid advancement of 5G/6G networks and cloud services led to a significant increase in cyber-attack data flow, making real-time identification and consistent detection increasingly difficult [10].

Generative Adversarial Networks (GANs) are flexible methods for generating high-quality data and classifying information. They consist of a generator and a discriminator to detect anomalies in time series data. GANs have become prevalent due to their ability to handle imbalanced datasets through adversarial learning principles. Motivated by this, we propose a Low-rate Denial of service Attack Detection and Mitigation (LDADM) framework to detect LDoS attacks. The model trains over epochs, generates sequences with predetermined length, step size, and batch size, and obtains an anomaly score. An LDoS attack is detected if the score

is above a predefined threshold. After detecting an attack, it filters the attack traffic based on mitigation measures. These measures ensure temporary blocking and prevent unwarranted disruption of valid traffic. The contributions of this paper are as follows:

- 1) We propose a framework that utilizes GANs for identifying LDoS attacks in a real-time SDN environment.
- 2) We develop an efficient mitigation technique to prevent detected LDoS attacks before they cause significant damage. The temporary blockage of harmful IPs and the control of their removal after a specific amount of time permits legitimate communication.
- 3) We conducted comprehensive experiments to evaluate the effectiveness of LDADM. It achieves high accuracy in real-time attack detection and mitigation, outperforming existing techniques.

We organize the rest of this work as follows: Section II offers an overview of SDN, LDoS attacks, and GAN, encompassing the core principles and background details. Section III provides an overview of the existing research in the field. Section IV introduces the proposed system and methodologies. Section V contains comprehensive information regarding the experimental configuration and the procedure employed to create the dataset. It also presents the results of the experiment and assesses the model’s performance using those outcomes and Section VI functions as the conclusion of the entire work.

II. BACKGROUND

A. Software-Defined Networking

The architecture of SDN is illustrated in Figure 1. It consists of three planes: the control plane, the data plane, and the application plane. The control plane is responsible for the management of data packet routing within the network. The SDN controller functions as the network’s core decision-making authority for routing and establishing communication with network devices such as switches and routers. The forwarding devices that make up the data plane are responsible for processing, forwarding, and aggregating network packets. The application plane encompasses a variety of network applications and services. The application plane aims to create and implement network services that can adapt to changing conditions, improving overall efficiency and adaptability.

When the switch gets a packet header that does not match with the existing flow rules, it initiates `packet_in` and `packet_out` events. The SDN controller transmits flow rules to switches using the OpenFlow protocol through the southbound interface. Applications communicate with the SDN controller via the northbound interface to request network resources or provide services such as network administration, security, and traffic monitoring.

B. LDoS Attacks

LDoS attacks target network protocols and routing queue management systems by sending small amounts of traffic to a TCP/IP network, disrupting its regular operation. LDoS attacks are identifiable by the regular occurrence of frequent pulses; hence, they are also called pulsing attacks [11]. These attacks

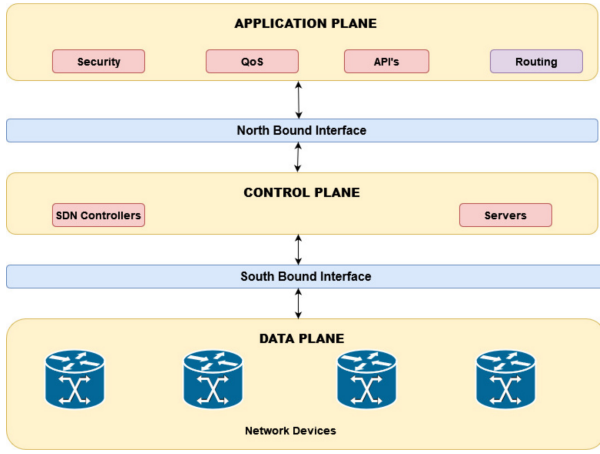


Fig. 1. Software-defined networking architecture.

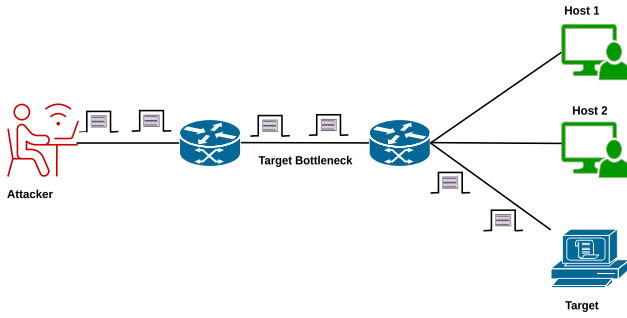


Fig. 2. LDoS attack scenario.

interfere with harmless TCP flows by employing square wave pulses, as shown in Figure 2. The size and duration of the pulses determine the severity of the attack's effect. Each pulse measures the attack traffic over time.

Commonly used Internet systems are the primary target of LDoS attacks. They aim to lower the rates at which specific protocols or applications consume resources. LDoS attackers intentionally direct valid traffic toward the target. This prompts the target's system to generate adverse responses, thereby disrupting or deteriorating service quality. Currently, LDoS attacks mostly focus on systems that include centralized services and devices, such as cloud computing and big data service platforms.

LDoS attacks generate traffic that displays intermittent patterns and maintains a consistently low average rate. This makes them a greater threat to SDN compared to traditional networks. A rapid surge in attack traffic overwhelms the victim's switches, causing packet loss in regular TCP flows and requiring repetitive retransmission cycles. Attackers often leverage network measurement tools [12], [13] to analyze bottleneck links, which are narrow bandwidth lines used for both control and data traffic. The attacker intentionally sends attack data traffic to overload the bottleneck buffer. This leads to a significant increase in the usage rate of the connection and causing severe disruption to network services. Identifying LDoS attacks is crucial because they pose a significant danger to both network operators and end users.

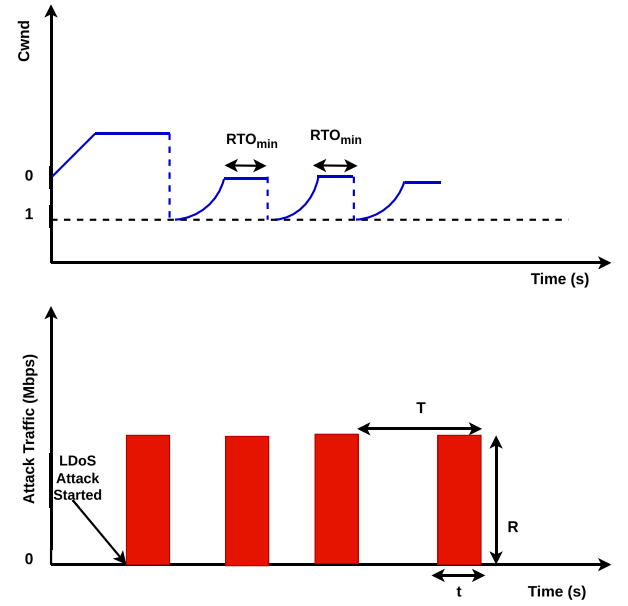


Fig. 3. LDoS attacks targeting RTO mechanism.

C. TCP-Targeted LDoS Attacks

LDoS attacks specifically target two TCP characteristics: Retransmission TimeOut (RTO) and Additive Increase Multiplicative Decrease (AIMD). The RTO mechanism is a congestion control approach in TCP that functions by utilizing a window established at the data sender. The congestion window regulates the amount of data that a network can transmit before requiring an acknowledgment from the recipient. Every time a TCP sender sends a message, the RTO mechanism starts a timer. When the timer expires without an acknowledgment, it indicates that the message may not have reached the receiver due to packet loss or network issues, prompting the sender to retransmit. This indicates that there is a high level of network congestion, which requires the TCP protocol to switch to the slow-start state. That is, if an acknowledgment does not arrive within the specified delay period, it resets the congestion window to 1, initiating a slow start phase for retransmission. In real-world scenarios, the RTO is adjusted based on the Round-Trip Time (RTT). It often ranges from 10 to 100 milliseconds in real networks. It is computed as shown in (1), where G represents the time scale, $SRTT$ represents the smoothed RTT, and $VRTT$ represents the smoothed RTT variation. Usually, the RTT and $SRTT$ values, including G and four times the $VRTT$, are extremely low. Therefore, according to (1), the value of RTO is often equivalent to RTO_{min} .

$$RTO = \min(RTO_{max}, \max(RTO_{min}, SRTT + \max(G, 4 \times VRTT))) \quad (1)$$

LDoS attacks exploit the vulnerability of TCP congestion management systems, causing TCP senders to repeatedly switch between congestion and recovery states. While using the RTO method, attackers send LDoS pulses during each retransmission event. This stops retransmissions and keeps the TCP congestion window (cwnd) value constant. The attacker begins an attack pulse that lasts for a specific duration, as shown in Figure 3. The attacker then waits for a retransmission

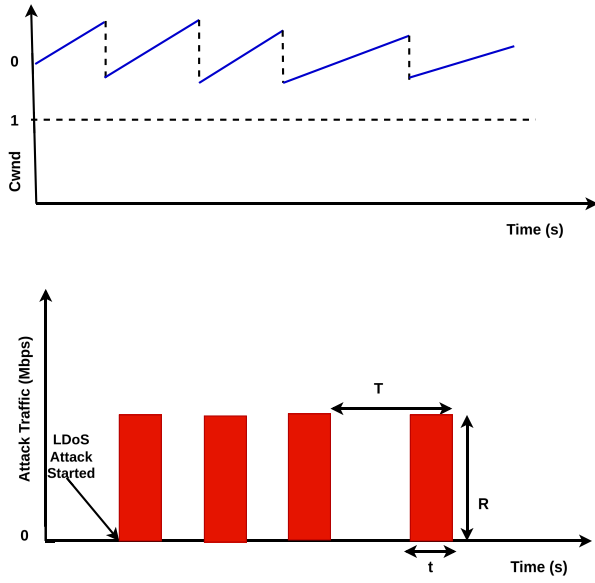


Fig. 4. LDoS attacks targeting AIMD mechanism.

time out before initiating another pulse. The attacker exhibits this behavior with the intention of causing the victim network to exit the RTO state. This allows for a more efficient attack while maintaining the unchanged attack cycle.

TCP uses the AIMD as a mechanism to control network congestion. When the network is not experiencing congestion, AIMD gradually increases the cwnd. When a packet loss is detected, AIMD multiplicatively decreases the cwnd. The balance between aggressive bandwidth usage and conservative load avoidance guarantees equity among competing TCP flows. Several TCP versions and other network protocols extensively employ AIMD to ensure efficient and stable data delivery throughout networks. Equations (2) and (3) describe the principle of AIMD, where I represents the additive increase of cwnd when ACK packets are received within an RTT. W_t is the value of cwnd at time t , R represents RTT, and W_{t+R} represents the value of cwnd after one RTT. D represents the multiplicative decrease, and α , β are the parameter values typically set to 1 and 0.5, respectively [14].

$$I: W_{t+R} \leftarrow W_t + \alpha, \quad \alpha > 0 \quad (2)$$

$$D: W_{t+R} \leftarrow \beta \times W_t, \quad 0 < \beta < 1 \quad (3)$$

While using the AIMD mechanism, attackers send periodic bursts to continuously decrease cwnd, reducing it to half its previous size. When attacks are silent, cwnd increases by 1 per RTT and then suffers new bursts, as illustrated in Figure 4. The recipient provides congestion feedback in the form of duplicate ACKs. The sender then uses this information to reduce the congestion window in accordance with the congestion avoidance method.

Both the RTO and AIMD mechanisms control the congestion window of a TCP sender on real networks. Therefore, the consequences of an attack vary between these two effects. During the attack, there may be instances where it leads to a timeout and RTO effect, while in other cases, it may produce a

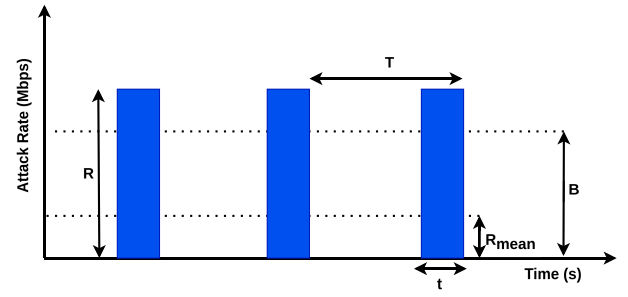


Fig. 5. LDoS attack model.

decrease in the congestion window. The intensity of the attack and network congestion determine this variation.

D. LDoS Attack Model

To assess the identification and countermeasures for LDoS attacks, it is important to create an LDoS attack model and establish its attack attributes. The LDoS attack model consists of three essential parameters: attack cycle, attack pulse length, and attack rate. Adjusting these parameters may change the attack's impact and limit the amount of traffic transmitted during each cycle to stay below bandwidth capacity. The LDoS attack model is depicted in Figure 5. T represents the attack cycle, t represents the duration of an individual attack pulse, R represents the attack rate of a single attack, B represents the bottleneck bandwidth, and R_{mean} indicates the average attack rate within one attack cycle.

To initiate an LDoS attack, it is essential to ensure that the rate of data transmission (R) exceeds the capacity of the bottleneck bandwidth, as depicted in (4). This will overload the targeted connection. However, it is essential that the total amount of attack data transferred during each attack period, known as Attack Cost (AC), stays below the bandwidth capacity, as indicated in (5).

$$R > B \quad (4)$$

$$AC = \frac{R \times t}{\text{Bandwidth} \times T} < 1 \quad (5)$$

E. Generative Adversarial Networks (GAN)

A Generative Adversarial Network (GAN) is a learning model that is capable of operating in unsupervised, supervised, and reinforcement learning contexts. The model comprises a generator (G) and a discriminator (D) that learn concurrently. The generator produces samples sufficiently realistic to fool the discriminator. The discriminator differentiates between real and fake data. Figure 6 illustrates the architecture and training process of a GAN. The model transforms raw data (z) into synthetic data $G(z)$, with the discriminator classifying the data sources as genuine or fake based on the distribution of the real data. The generator seeks to create synthetic data that closely resembles real data (x), which is evaluated by the discriminator. Iterative tuning improves performance, and once D's discriminative ability reaches a threshold, G can deduce that it understands the genuine data distribution. Back Propagation can adjust the parameters of both the generator

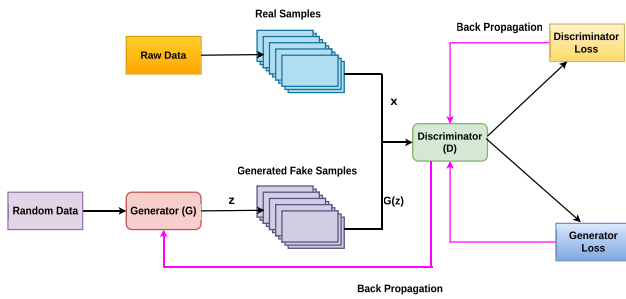


Fig. 6. The structure of GANs.

and discriminator, improving their functions and generating realistic data.

III. RELATED WORK

This section provides an overview of the literature on detection approaches for DDoS and LDoS attacks in SDN, along with GAN-based approaches to detect and mitigate them.

A. DDoS Attack Detection Approaches

Wang et al. presented a defense system to detect and mitigate DDoS attacks in SDN by employing hybrid deep learning techniques and IP traceback [15]. The authors of [16] presented a framework to enhance high-speed network intrusion detection and address DDoS attack challenges in SDN by utilizing the Data Plane Development Kit (DPDK). Ling et al. introduced a real-time DDoS detection technique based on the hurst index [17]. In [18], a mechanism was introduced to identify and mitigate DoS attacks in SDN by reducing the rate of control packets and retransmission during attacks. A DDoS detection framework based on deep neural networks was presented in [19]. The authors of [20] investigated advancements in DDoS attack detection and future directions for improved security in SDN environment. A collaborative source-side DDoS attack detection framework using Long Short-Time Memory (LSTM) to address the limitations of target-side detection techniques was presented in [21].

B. LDoS Attack Detection Approaches for SDN

An algorithm that utilizes source IP addresses to detect real-time LDoS attacks by monitoring traffic fluctuations was presented in [22]. The authors of [23] investigated a Gramian angular summation field transformation-based LDoS detection and mitigation framework for SDN. An adaptive Kohonen network-based fine-grained LDoS detection model that leverages the Smith-Waterman algorithm and cluster analysis was presented in [24]. An online monitoring and mitigation architecture was developed for SDN that efficiently prevents LDoS attacks without requiring any changes to switches or protocols [25]. The study in [26] presented a framework that uses histogram-based gradient boosting and peak detection to detect and mitigate LDoS attacks in real time. In [27], a real-time framework was introduced to identify and mitigate LDoS attacks in SDN. It uses machine learning to monitor traffic

performance and evaluate flow features. A mitigation system for SDN that uses decision trees and dynamic sliding window algorithms to protect against LDoS attacks was presented in [28]. A security solution that effectively counters low-rate TCP attacks by monitoring data transfer rates was developed in [29].

An online attack detection and mitigation system for SDN that uses traffic features and a Lightgbm classifier was introduced in [14]. Liu et al. presented a real-time anomaly protection framework to detect and reduce TCP-targeted LDoS attacks in SDN [30]. A two-phase detection system was developed by combining lightweight detection in the data plane with in-depth identification using Bayesian voting in the control plane [31]. An LDoS attack detection algorithm that integrates an optimization-based heuristic algorithm with a backpropagation neural network was presented in [32]. Chen et al. introduced an online real-time LDoS detection system that employs CNN and dynamic time warping (DTW) algorithms [33]. Liu et al. developed a two-step self-adjusting support vector machine to identify LDoS attacks [34]. Deng et al. introduced a system to detect real-time LDoS attacks in SDN environments by leveraging the Extreme Gradient Boosting technique [35]. An ensemble model to identify LDDoS threats by utilizing CNN and a bidirectional LSTM network was presented in [36].

Existing LDoS mitigation frameworks in [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26], [27], [28], [29], [30], [31], [32], [33], [34], [35], [36] employ a multi-stage process to identify the victim and trace the attacker. Then, they employ port-level or flow-level drop restrictions to mitigate the attacks. However, these solutions have limitations such as, high computational overhead, delayed mitigation, and inability to adapt to new LDoS strategies. Further, aggregate port/flow-based action could cause collateral damage to legitimate traffic. In contrast, the proposed mitigation strategy is a lightweight, flow-aware, and adaptive methodology. It implements targeted mitigation at the flow level ($\text{src_ip} \rightarrow \text{dst_ip}$) immediately upon receiving an attack alert. This enables fine-grained control without affecting legitimate traffic. Further, the proposed strategy employs a time-restricted blocking method that automatically unblocks flows after a specified duration. This approach improves flexibility and prevents legitimate sources from permanently being disrupted.

C. GAN-Based Attack Detection Approaches

The study in [37] investigated the impact of data imbalance on DDoS attack detection. It also demonstrated that using synthetic GAN-generated data to balance the dataset significantly improves classification accuracy. A GAN-based approach with dual discriminators to detect adverse DDoS traffic was presented in [38]. Mustapha et al. designed an architecture for DDoS detection that uses Wasserstein GAN (WGAN) to generate adversarial attacks [39]. The authors of [40] introduced an unsupervised framework that utilizes WGAN with Gradient Penalty (WGAN-GP) and an encoder for DDoS detection and mitigation in SDN. The work in [41] employed GANs to generate realistic IoT traffic data

TABLE I
COMPARATIVE ANALYSIS OF GAN-BASED ATTACK DETECTION
APPROACHES WITH THE PROPOSED WORK

Work	Attack type		SDN-based	Detection		Mitigation
	DDoS	LDoS		Offline	Online	
[37]	✓	✗	✗	✗	✗	✗
[38]	✓	✗	✗	✓	✗	✗
[39]	✓	✗	✓	✓	✗	✓
[40]	✓	✗	✗	✓	✗	✗
[41]	✓	✗	✓	✗	✗	✗
[42]	✓	✗	✓	✓	✗	✗
[43]	✓	✗	✗	✗	✗	✗
[44]	✓	✗	✗	✗	✗	✗
[45]	✓	✗	✗	✓	✗	✗
[46]	✓	✗	✗	✗	✗	✗
[47]	✓	✗	✓	✓	✗	✗
[48]	✓	✗	✓	✓	✗	✓
[49]	✓	✗	✗	✗	✗	✗
Proposed	✓	✓	✓	✓	✓	✓

for DDoS attack detection in smart home environment. A Conditional GAN (CGAN)-based DDoS detection technique was developed for SDNs by utilizing conditional training and flow table features [42]. Abdelaty et al. introduced an adversarial training method based on WGAN-GP [43]. Their method improved system robustness and reduced undetected malicious flows by generating fake benign samples that disrupt real attacks.

Shieh et al. employed CycleGAN to generate realistic malicious traffic [44]. They highlighted the threat posed by adversarial DDoS attacks capable of bypassing ML/DL-based detection. A privacy-preserving framework for DDoS detection in multi-domain networks by employing Federated Learning and GAN was presented in [45]. Saka et al. investigated a Conditional Tabular GAN (CTGAN), Tabular Variational Autoencoder (TVAE), and CopulaGAN for the synthesis of tabular data to improve DDoS detection [46]. The work in [47] combined Gated Recurrent Units (GRU) with a GAN discriminator to detect volumetric DDoS attacks in SDN. The authors of [48] presented a GRU-enhanced GAN that uses flow and entropy-based features for unsupervised DDoS detection in SDN. Chauhan and Shah Heydari introduced a GAN-based methodology in [49] for generating polymorphic adversarial DDoS attacks that can bypass ML-based Intrusion Detection Systems (IDSs).

The comparative analysis of existing GAN-based attack detection approaches with the proposed LDADM approach is presented in Table I. Although prior work in [37], [38], [39], [40], [41], [42], [43], [44], [45], [46], [47], [48], [49] employed GANs, none of these works investigated the GAN-based model for the detection and mitigation of real-time LDoS attacks for SDN environment. Most of the existing works focus on conventional DDoS attacks instead of stealthy LDoS attacks. Various studies are implemented in non-SDN environments that lack centralized control and programmability for adaptive mitigation. Most existing studies employ offline detection techniques, hence limiting their effectiveness in dynamic, real-time networks. Furthermore,

few studies address mitigation, often treating it as a post-processing task or depending on external systems such as firewalls, IDS, etc., rather than incorporating it into the detection pipeline. This separation limits quick responses to LDoS attacks. In contrast, the proposed LDADM framework is an SDN-native solution that integrates real-time detection and mitigation. It employs a GAN-based model for LDoS attack detection and incorporates dynamic flow rule management within a controller to implement rapid mitigation actions.

IV. PROPOSED SYSTEM AND METHODOLOGY

This section presents LDADM, a framework for the real-time detection and mitigation of LDoS attacks. It also provides a detailed discussion of GAN-based algorithms used for the efficient detection and mitigation of LDoS attacks.

Figure 7 illustrates the proposed LDADM framework for detecting and mitigating LDoS attacks in an SDN environment. The framework consists of three layers: the data layer, the SDN controller, and the application layer. The data layer processes both legitimate user traffic and malicious attack traffic from OpenFlow-enabled switches or users. It also collects flow data and transmit it to the SDN controller for analysis. The SDN Controller serves as the central intelligence, consisting of two main modules: detection and mitigation. The controller utilizes the southbound interface to get flow statistics such as packet counts, byte counts, and flow durations, etc., from switches. These flow statistics are transmitted to the detection module of the controller. This module performs comprehensive traffic analysis to transform raw data into traffic features. These features undergo extraction and selection for the training of the GAN model. The trained GAN model analyzes incoming real-time traffic to determine whether each flow is normal or malicious, producing detection results. Upon detection of an attack, control moves to the mitigation module. It records the detected malicious flows, determines the attacker's IP address, identifies the victim, and develops mitigation strategies. The mitigation strategy employs a time-restricted blocking mechanism that automatically unblocks flows after a specified duration. It implements flow rules through the northbound interface, which are then deployed to the switches for enforcement. The application layer interfaces with external systems using REST APIs. It provides detection and mitigation results while enabling dynamic network monitoring and control.

A. Attack Detection Module

The attack detection module includes two algorithms: GAN-based model training and real-time attack detection in SDN environments.

Algorithm 1 illustrates the training of GAN-based model to detect LDoS attacks in real-time environments. It takes as input data comprising traffic sequences, sequence length seq_len , step size, number of epochs and batch size. The algorithm begins by preprocessing the input data and initializes an empty list of *sequences*. It iteratively extracts data segments

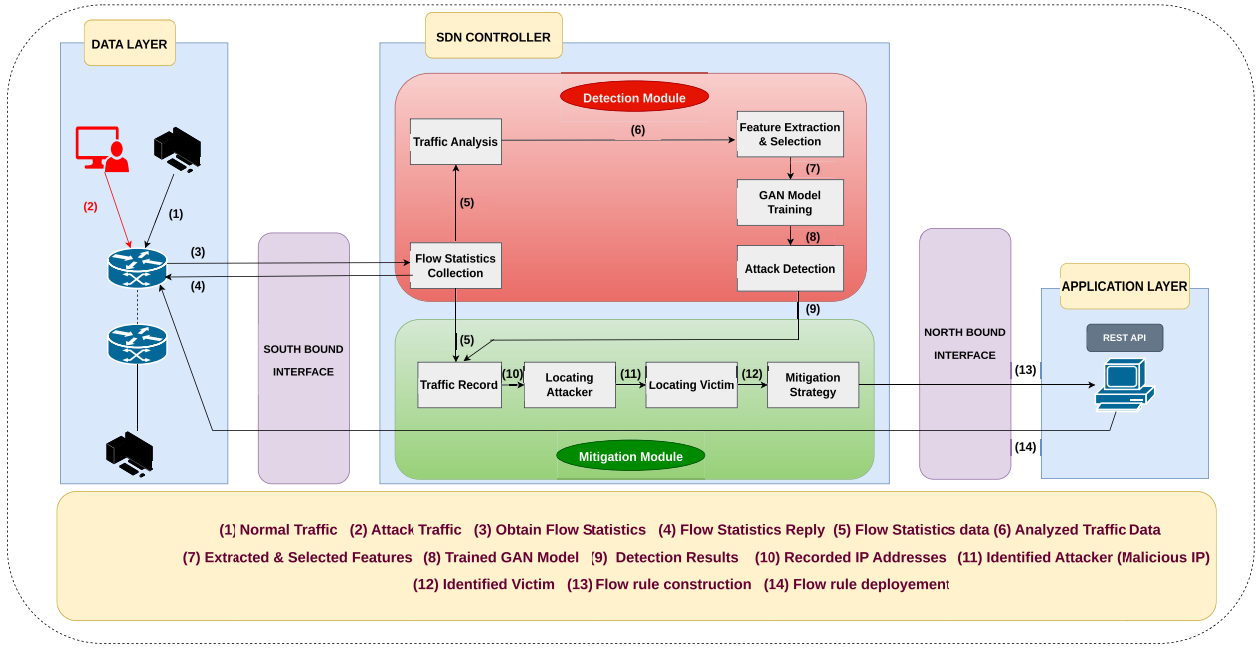


Fig. 7. LDADM framework.

Algorithm 1 GAN Model Training**Input:** data, seq_len, step, epochs, batch_size**Output:** Discriminator, Generator1: **Preprocess data**2: Initialize sequences $\leftarrow []$ 3: **for** $i \leftarrow 0$ **to** $len(data) - seq_len$, **step** **do**4: temp $\leftarrow data[i: i + seq_len]$

5: Append temp to sequences

6: **end for** **Construction of GAN:**7: Generator $\leftarrow build_generator()$ 8: Discriminator $\leftarrow build_discriminator()$

9: Compile and evaluate GAN by combining Generator and Discriminator

Training of GAN:10: **for** epoch $\leftarrow 1$ **to** epochs **do**11: **for** each batch in sequences **do**

12: Train Discriminator on real and fake sequences

13: Train Generator

14: Compute loss for both Discriminator and Generator

15: Update Discriminator and Generator

16: **end for**17: **end for**18: **return** Discriminator, Generator

of length seq_len using the specified step size from raw data and adds them to the *sequences* list. This procedure transforms the continuous data stream into fixed-length segments for better analysis over time. The seq_len parameter defines the number of time steps or observations in each input sequence. This parameter also defines the volume of data that the model concurrently evaluates for generating predictions or finding

abnormalities. The step parameter ensures that the sequences overlap by one time step. This overlap allows the model to identify deeper changes and patterns in the data. After preparing the sequences, the algorithm constructs the GAN model, which comprises a generator that creates synthetic sequences and a discriminator that differentiates between real and fake sequences. The GAN model undergoes a training phase for a predefined number of epochs. For each batch in the sequence, the model trains discriminator (D) and generator (G). The batch size parameter denotes the number of sequences that the model concurrently processes in each iteration. The training begins with the generator producing fake data using random noise sampled from latent space. The discriminator is evaluated on the real data from the *sequences* and fake data produced by the generator. Then, loss functions are computed for discriminator and generator networks. Finally, the generator and discriminator networks are updated through backpropagation. The hyperparameters utilized in the algorithm are presented in Table II.

Algorithm 2 defines a real-time LDoS attack detection mechanism employing a discriminator to categorize traffic as either legitimate or malicious depending on a predetermined threshold. It requires a real-time flow for assessment, a discriminator to classify the flow as either legitimate or an attack, and a threshold.

The algorithm begins with the extraction of features from the input data flow. The feature vector is then passed to the discriminator, which computes a score to determine the probability that the input flow is legitimate or an attack. If the score exceeds a predetermined threshold, it is categorized as an attack. If it is below or equal to the threshold, it is categorized as legitimate traffic. The *attack_flag* is set to 1 upon detection of an attack and 0 when the traffic is considered to be

TABLE II
HYPERPARAMETERS

Hyperparameter	Values
Sequence Length (<i>seq_len</i>)	10
Step Size (step)	1
Number of Epochs (epochs)	30
Anomaly Detection Threshold (η)	0.5
Optimizer	Adam
Learning Rate	0.0002
Beta	0.5
Batch Size (batch_size)	64
Generator Layers	4
Discriminator Layers	3
Activation Function	sigmoid

Algorithm 2 GAN-Based Attack Detection Algorithm

Input: flow, Discriminator, threshold

Output: attack_flag

```

1: feature_vector ← extract_features(flow)
2: score ← Discriminator(feature_vector)
3: if score > threshold then
4:   Print "Attack detected from source IP"
5:   attack_flag ← 1
6: else
7:   Print "Legitimate traffic detected from source IP"
8:   attack_flag ← 0
9: end if
10: return attack_flag

```

legitimate. The algorithm's final result is the *attack_flag*, indicating the identification of an LDoS attack.

B. Mitigation Module

Algorithm 3 describes the mitigation strategy to counter LDoS attacks through the use of defensive measures against malicious IP addresses. It takes as input a traffic flow, a specified block duration, and the outcome of the detection algorithm on the traffic flow, which is represented as an attack flag. The algorithm initializes an empty list of *malicious_ips* to store the IP addresses of sources identified as malicious. If the *src_ip* of the traffic flow detected as an attack is not present in the *malicious_ips* list, then it is appended to the list and records the action as blocking the *src_ip* address. The algorithm then creates a flow rule that aims to prevent packets from the malicious IP. The match field of the flow rule corresponds to packets with *ipv4_src* specified as *src_ip* and *eth_type* specified as IPv4. The corresponding action for these matched packets is to drop them. The *add_flow_entry()* method incorporates the flow entry into the network switch. This method requires parameters including the match field, priority, action, and the datapath object. The algorithm generates a message confirming that the *src_ip* is blocked for the designated block duration. If the *src_ip* is already included in the *malicious_ips* list, then the algorithm alerts that the IP is currently blocked. The algorithm continually updates the

Algorithm 3 Mitigation of LDoS Attacks

Input: Traffic sequence flow, block_duration, detection result attack_flag

Output: Defense Rule

```

1: Initialize malicious_ips ← []
2: Function: handle_mitigation()
3: if attack_flag == 1 then
4:   if src_ip not in malicious_ips then
5:     Log "Blocking IP: src_ip"
6:     Add src_ip to malicious_ips
7:     Construct flow rule to block packets from src_ip
8:     Match ← {ipv4_src = src_ip, eth_type = IPv4}
9:     Action ← Drop
10:    add_flow_entry(Match, priority, actions, datapath)
11:    Print "src_ip is blocked for block_duration seconds"
12:    Wait untill block_duration to release src_ip later
13:  else
14:    Print "src_ip is already blocked"
15:  end if
16: end if
17: Function: release_blocked_ips()
18: for each src_ip in malicious_ips do
19:   if block_duration has expired for src_ip then
20:    Remove src_ip from malicious_ips
21:    Print Released src_ip
22:   end if
23: end for
24: return Action

```

list of blocked IPs by releasing IPs from the *malicious_ips* list after the expiration of their blocking period. That is, the algorithm traverses all blocked IPs and verifies whether the block duration for an IP is lapsed. It removes the *src_ip* lapsed blocking period from the *malicious_ips* list and records the release of the *src_ip*. Finally, the algorithm produces an action that signifies the implementation of a defense rule to mitigate identified malicious activities. This algorithm ensures the temporary blocking of harmful IPs, thereby preserving the network's operational integrity and protecting it from potential LDoS attacks.

V. RESULTS AND ANALYSIS

This section presents the experimental setup, traffic generation, evaluation metrics, evaluation of LDADM framework, and performance comparison of the LDADM with existing methods.

A. Experimental Setup

The LDADM is evaluated using Linux operating system and SDN environment created by Mininet [50] and Ryu controller as illustrated in Table III. We considered the Abilene topology from the Internet Topology Zoo [51] as shown in Figure 8. It consists of 25 links, 11 switches, and 11 hosts. We have obtained the topology as a graphic file with. gml

TABLE III
SIMULATION ENVIRONMENT

Environment Configuration	Details
Operating System	UBUNTU 20.04.4 LTS
CPU	Intel Core i7-6700 @ 3.40 GHz
RAM	64 GB
HDD	1 TB
Python	Python 3.8.10
SDN Controller	RYU 4.34
Traffic Generator	Iperf 3.7 (cJSON 1.5.2)
Network Emulator	Mininet 2.3.0d6
Packet Analyzer	Wireshark 3.2.3
Network Interface	Ethernet Connection

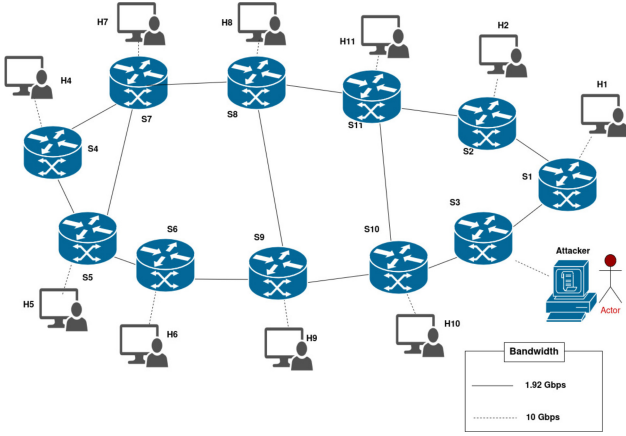


Fig. 8. The network topology.

extension and used Python code to convert it into Mininet file with. mn extension. The topology uses OpenFlow v1.3 and OpenVSwitches that implement the OpenFlow protocol (OpenVSwitch v2.5.5). The bandwidth between switches is set to 1.92 Gbps, while the bandwidth between a switch and a host is set to 10 Gbps. All switches are interconnected by a solitary remote Ryu controller. The IP addresses of all network devices are configured within the 10.0.0.1/11 network segment, ensuring connectivity between hosts.

B. Traffic Generation

The background traffic is generated by the iperf3 tool and LDoS attack traffic is generated through a Python script. The traffic collection and recording module interacts in real time with switches connected to the Ryu controller to obtain flow table data. The module reads incoming messages and extracts relevant information.

C. Evaluation Metrics

The LDoS attack detection and mitigation framework categorizes time periods as positive samples during attacks and negative samples during normal network conditions. True positives (TP) are the number of accurately detected positive samples, whereas false positives (FP) are negative samples that are incorrectly labeled as positive. False negatives (FN) are positive samples that are incorrectly categorized as negative, whereas true negatives (TN) are accurately detected negative

samples. To measure the effectiveness of mitigation, we utilized six metrics: accuracy, precision, recall, F1-score, false positive rate (FPR), and response time (RT).

1) *Accuracy*: This metric measures the degree to which detection results correspond to true values. It is calculated as follows:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (6)$$

2) *Precision (P)*: It computes the percentage of positives that are accurately detected during LDoS attacks. It is expressed as follows:

$$P = \frac{TP}{TP + FP} \quad (7)$$

3) *Recall*: It is a measure of the proportion of true positives correctly detected. It is calculated as follows:

$$\text{Recall} = \frac{TP}{TP + FN} \quad (8)$$

4) *F1-Score*: This metric measures the balance between precision and recall in a classification model. It is computed by obtaining the harmonic mean of the precision and recall values of the model. It is calculated as follows:

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (9)$$

5) *False Positive Rate (FPR)*: It estimates the percentage of negative cases that are mistakenly identified as positive. It is determined as follows:

$$\text{FPR} = \frac{FP}{TN + FP} \quad (10)$$

6) *Response Time (RT)*: This metric measures how quickly a system detects and reacts to an attack. It is calculated as follows:

$$RT = \text{MitigationTime} - \text{AttackStartTime} \quad (11)$$

D. Comparison Approaches

The performance of the LDADM is compared with three state-of-the-art approaches: PeakSAX [25], HGB-FP [26], and GASF-IPP [23]. We briefly describe these approaches below.

- 1) *PeakSAX*: This approach uses a traffic symbolization technique to detect LDoS attacks and classifies attacks based on peak attributes using a random forest classifier. Then, mitigation is performed by blocking malicious traffic using flow rules.
- 2) *HGB-FP*: This framework employs a histogram-based gradient boosting for the detection of LDoS attacks. The mitigation process involves applying the peak finding technique and blocking malicious traffic by implementing flow rules.
- 3) *GASF-IPP*: This approach identifies LDoS attacks using the Gramian angular summation field transformation technique. It mitigates attacks by using IP-Port analysis and implementing flow rules to block malicious traffic.

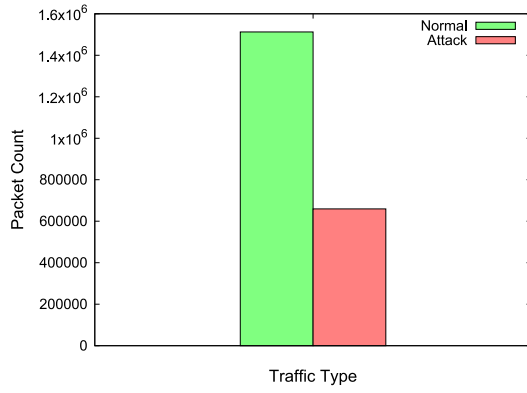


Fig. 9. Traffic distribution of the generated dataset.

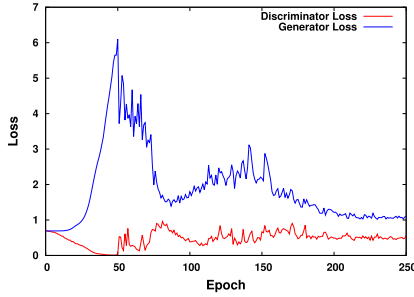
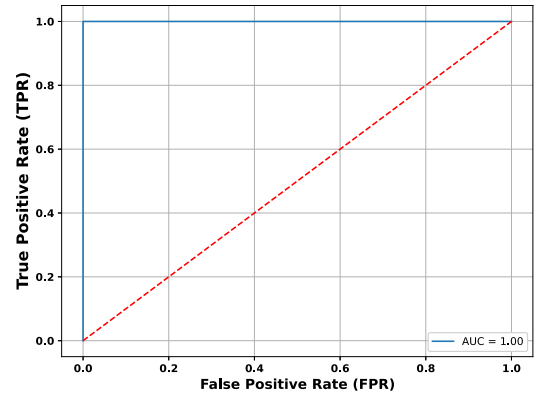


Fig. 10. Generator loss vs discriminator loss.

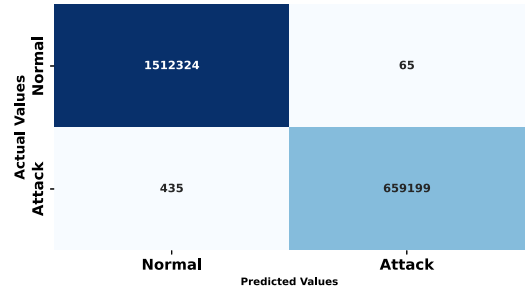
E. Performance Evaluation

The distribution of traffic within a dataset is crucial for understanding network traffic balance, particularly for the detection of anomalies and performance evaluation. Figure 9 illustrates the traffic distribution in our generated dataset. It consists of 21,71,852 total values with 15,12,389 representing normal traffic and 6,59,463 indicating attack traffic. A majority of normal traffic results in an imbalance. Class imbalance significantly affects the accuracy and efficacy of LDoS attack detection. The proposed LDADM framework manages class imbalance by prioritizing the modeling of normal traffic instead of requiring large volumes of attack data. During training, the generator learns to mimic legitimate traffic patterns and the discriminator is able to distinguish between real and generated data. This process enhances the discriminator's ability to identify normal traffic, enabling it to identify LDoS attacks even when these occurrences are rare or new. Instead of generating fake attack samples or oversampling the minority class, the proposed LDADM framework trains its discriminator to thoroughly comprehend normal behavior. Focusing on normal traffic patterns reduces the risk of bias and overfitting, leading to more robust and reliable detection of LDoS attacks.

Figure 10 represents the training behavior of the proposed LDADM in terms of discriminator loss and generator loss. These losses represent the specific functions optimized by the discriminator and generator, respectively. These losses are vital for maintaining a balance between the generator and discriminator. They force the generator to produce realistic data and test the discriminator's ability to differentiate between real and generated inputs. The discriminator loss measures



(a) ROC curve



(b) Confusion Matrix

Fig. 11. Visual summary of classification metrics.

how well the discriminator can classify inputs as real or fake. It is often improved using binary cross-entropy to help the discriminator learn to differentiate between real and generated data. As the discriminator gets better at distinguishing real and fake inputs, it supplies more accurate feedback to help the generator create more realistic synthetic data. The generator loss measures how well the generator can create synthetic data that misleads the discriminator, pushing the generator to produce outputs that look like real data. The generator is designed to produce samples that the discriminator erroneously identifies as real, hence enhancing the reality of the generated data over successive iterations. This adversarial relationship is essential to GAN training.

The loss patterns exhibited three distinct phases of learning during training. In the initial phase (epochs 0–50), the generator's loss increased, while the discriminator's loss decreased. This signified the discriminator's initial dominance, as the generator at this stage produced inconsistent traffic patterns that the discriminator could easily identify as fake. During the intermediate phase (epochs 50–200), the generator and discriminator maintained a balanced and competitive adversarial dynamic, exhibiting fluctuations in their loss patterns. During the final phase of convergence (epochs 200–250), both networks reached a state of relative equilibrium, with their losses stabilizing and fluctuations becoming minimal. The reduced volatility at this stage indicates the convergence of the adversarial training process, suggesting that additional training is likely to yield only minimal qualitative enhancements.

Figure 11(a) represents the Receiver Operating Characteristic (ROC) curve alongside its associated Area

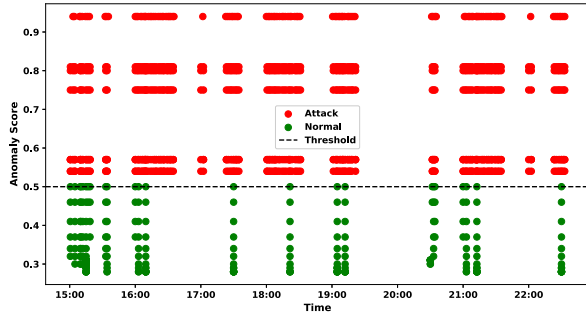


Fig. 12. Anomaly distribution.

TABLE IV
PERFORMANCE COMPARISON

Framework	Accuracy	Precision	Recall	F1-Score
GASF-IPP	0.9010	0.9103	0.9004	0.9100
HGB-FP	0.9101	0.9106	0.9122	0.9231
PeakSAX	0.9302	0.9109	0.9201	0.9303
LDADM	0.9998	0.9999	0.9996	0.9998

Under the Curve (AUC) value. The GAN model employs a neutral threshold of 0.5 to avoid sensitivity bias. The ROC curve demonstrates the network's predictive confidence, with elevated AUC values signifying effective class differentiation.

The confusion matrix illustrated in Figure 11(b) indicates that the model effectively identifies attack traffic. It shows a high number of accurately classified attacks and normal traffic samples. The model incorrectly categorizes a fraction of actual attacks as normal traffic, indicating the missed detection of some attacks. However, it incorrectly categorizes a fraction of normal traffic as an attack.

The distribution of anomaly scores depicted in Figure 12 illustrates the variation of anomaly scores over time. The neutral threshold is indicated in a horizontal black line, and an anomaly score beyond the threshold, indicated in red, signifies attack traffic. The lower scores indicated in green represent normal traffic.

It provides a clear differentiation between normal and anomalous classes, hence increasing the model's confidence and improving its accuracy.

1) *Detection Performance*: Table IV illustrates the detection performance of the LDADM in comparison to state-of-the-art approaches, namely PeakSAX [25], HGB-FP [26], and GASF-IPP [23]. The LDADM results in a 7.35%, 9.85%, and 10.96% higher accuracy compared to PeakSAX, HGB-FP, and GASF-IPP, respectively. The framework also achieves high precision by minimizing false positives and effectively managing dataset imbalance. It achieves a 9.77%, 9.80%, and 9.84% higher precision compared to PeakSAX, HGB-FP, and GASF-IPP, respectively. Moreover, the LDADM yields 8.64%, 9.82%, and 11.01% higher recall rates than PeakSAX, HGB-FP, and GASF-IPP, respectively, showing that it is better at finding attack traffic. Further, the LDADM achieves a 7.47%, 8.30%, and 9.86% higher F1-Score than PeakSAX, HGB-FP, and GASF-IPP, respectively.

TABLE V
ATTACK PARAMETERS SETTING

Set.No.	T(s)	t(s)	R(Mbps)	Cost ($\frac{R \times t}{T \times bw}$)
1	2	0.3	20	0.00156
2	2	0.2	25	0.00130
3	1	0.1	30	0.00156
4	2	0.1	30	0.00078
5	1	0.1	60	0.00312
6	2	0.1	60	0.00156
7	1	0.1	100	0.00520
8	2	0.1	100	0.00260

2) *Mitigation Performance*: To verify the effectiveness of the LDADM under different real-world conditions, we carry out eight sets of simulations under attacks with different parameters, as illustrated in Table V. It consists of attack cycle (T), burst duration (t), attack rate (R), attack time, and attack cost. The possible t values vary from 0.1 to 0.5, while the possible T values range from 1 to 5. Changes in attack parameters enable the framework to handle a wide range of attack patterns, reflecting real-world LDoS behavior. Table V also illustrates the influence of various attack parameters on the cost of an attack. As t or R increases, the cost increases, demonstrating that reduced values of t and R yield lower costs.

Figure 13 illustrates the network traffic patterns recorded during each simulation set executed with the LDADM deployed on the controller. Network traffic is recorded using Wireshark at a sampling period of 0.1 seconds. The blue traffic pattern denotes TCP traffic, whereas the green traffic pattern indicates UDP traffic. The red dotted line indicates the beginning of the attack, the green dotted line denotes the mitigation of the attack, and the gray region represents the network under attack.

We conduct experiments on each set of attack scenarios ten times and record the maximum and minimum response times, as illustrated in Table VI. It consists of Set No., R, t, T values, min, max, and average response times for eight attack scenarios. Note that the attack rate (R), burst duration (t), and cycle time (T) influence response times. The response time for all sets varies from 0.12 seconds to 21.39 seconds. Consequently, the LDADM can efficiently identify and mitigate LDoS attacks with an average response time of 8.58s.

Table VI shows that the order of response time depends on the combination of attack rate (R), burst duration (t), and cycle time (T). High attack rates, such as 60 to 100 Mbps, impose considerable load on the system, resulting in longer response times, whereas reduced attack rates, such as 20 to 30 Mbps, generate less traffic load, leading to faster responses. Reduced cycle times, like T = 1, lead to more frequent bursts, which gives the system less recovery time and a faster response than longer cycles, like T = 2. This is the case if the attack rates of any two attack scenarios are equal and also have the same burst times. Moreover, even though the attack rate of 20 Mbps is lower than 25 Mbps, longer burst durations (t = 0.3 s) make the system more vulnerable to attack traffic and result

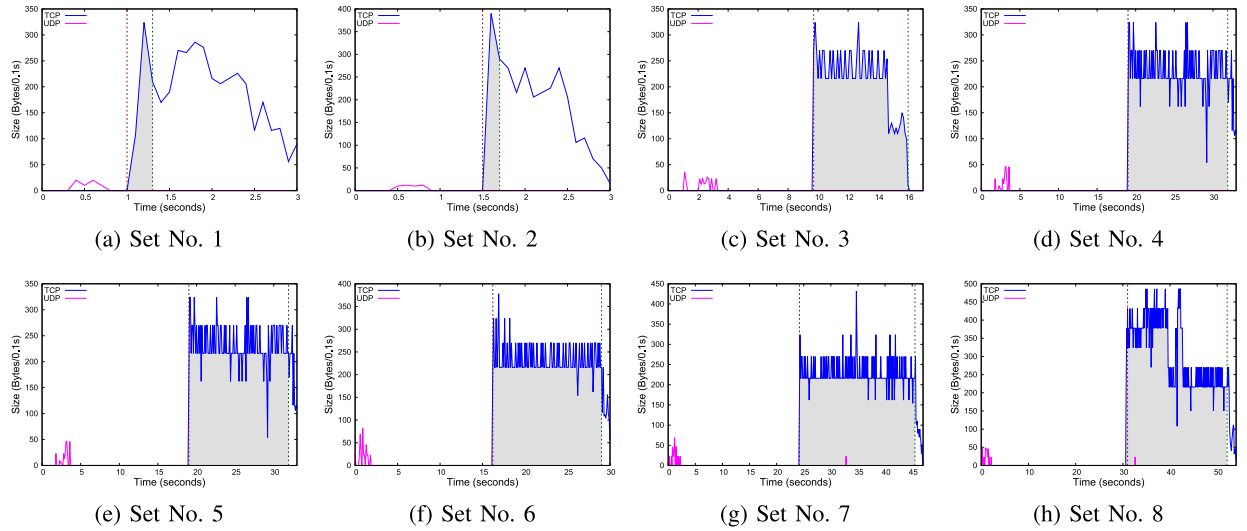


Fig. 13. Attack scenarios.

TABLE VI
MITIGATION RESPONSE TIMES

Set No.	(R,t,T)	Min (s)	Max (s)	Average (s)
1	(20,0.3,2)	0.13	0.26	0.16
2	(25,0.2,2)	0.13	0.22	0.15
3	(30,0.1,1)	5.26	6.44	6.06
4	(30,0.1,2)	5.88	6.38	6.20
5	(60,0.1,1)	6.83	12.89	10.99
6	(60,0.1,2)	11.72	12.96	12.30
7	(100,0.1,1)	10.14	21.35	15.13
8	(100,0.1,2)	12.66	21.40	17.69

in slower responses compared to shorter bursts ($t = 0.1$ s). These combinations demonstrate how quickly the LDADM can detect and respond to attacks.

Figure 14 illustrates the 95% confidence interval for the average response time of the LDADM compared to the PeakSAX, HGB-FP, and GASF-IPP approaches. The x-axis denotes the set number ranging from 1 to 8, whereas the y-axis represents the response times in seconds. Each model is denoted by a distinct color and the average response time is indicated by a dot. We can observe that the LDADM results in the lowest average response time compared to the state-of-the-art approaches. The average response time of the LDADM varies from 0.15 to 17.69 for different set numbers. The LDADM achieves a reduction in the response time of 17.89%, 21.71%, and 27.71% compared to PeakSAX, HGB-FP, and GASF-IPP, respectively. The confidence intervals for the LDADM are narrower, indicating improved stability and consistency in performance. This makes the framework quicker and more reliable for the evaluated scenarios, outperforming PeakSAX, HGB-FP, and GASF-IPP in response time and stability.

The state-of-the-art approaches, such as GASF-IPP and PeakSAX, utilized temporal compression methods and symbolic approximation approaches to handle class imbalance.

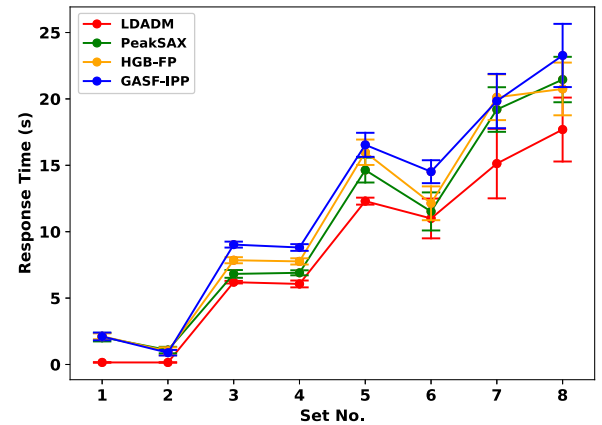


Fig. 14. Response time comparison.

The proposed LDADM framework employs a GAN to handle class imbalance. The GAN improves the discriminator's learning ability by producing synthetic minority-class attack samples. This preserves the statistical characteristics of actual traffic. Hence, facilitating the detection of complex LDoS patterns frequently ignored by the state-of-the-art approaches. Moreover, HGB-FP is limited by the absence of a generative process and dependence on manually engineered features. These constraints limit its adaptability in dynamic or imbalanced traffic conditions, limiting its effectiveness in addressing evolving network scenarios. The proposed LDADM framework is further differentiated by its seamless integration with the SDN control plane. This facilitates real-time flow-level feature extraction and rapid mitigation through dynamic flow rule updates, which is absent in GASF-IPP and PeakSAX. This significantly decreases detection latency and enhances response to evolving traffic conditions. We additionally integrate detailed statistical features, including entropy, inter-arrival time variance, and packet burst characteristics. This facilitates enhanced LDoS detection compared to the aggregated or symbolic information employed in state-of-the-art approaches. These architectural and algorithmic advancements together results

in the continuously improved performance evaluation metrics and faster mitigation response for the proposed LDADM framework.

3) *Resource Consumption*: Memory consumption refers to the amount of memory utilized by a program during its execution. The memory utilization of the proposed LDADM compared to state-of-the-art PeakSAX, HGB-FP and GASFI-IPP approaches is illustrated in Figure 15(a). The proposed LDADM framework exhibits marginally higher memory utilization when compared to the state-of-the-art approaches. Two deep neural networks, the generator and the discriminator, which are trained simultaneously, increase storage requirements of the proposed LDADM framework. However, this increase is marginal and well within acceptable operational limits. Specifically, the proposed LDADM framework exhibits average memory usage of 8.8%, which is only 1-2% higher than memory utilization of the state-of-the-art approaches. GASFI-IPP exhibits moderate memory utilization, with average memory consumption of 7.5%. HGB-FP demonstrates slightly lower consumption, with an average memory utilization of 6.8%. Finally, PeakSAX exhibits minimal resource utilization, with average memory consumption of 6.5%. The additional memory required by the proposed LDADM framework is feasible and can be managed by SDN controllers operating within standard resource constraints. The minor increase in memory utilization does not affect the controller's ability to manage network events or regulate flows. This indicates that enhanced detection capabilities can be achieved without compromising the overall stability of the system.

The CPU consumption of a certain process measures the extent to which the processor is utilized by that process. Figure 15(b) illustrates the CPU utilization of the proposed LDADM framework compared to the state-of-the-art PeakSAX, HGB-FP and GASFI-IPP approaches. The proposed LDADM framework exhibits marginally higher CPU utilization when compared to the state-of-the-art approaches. The synthetic samples generated in each training iteration of the proposed framework add extra CPU computation. The discriminator and generator are updated alternately also increases CPU workload. However, these increases are marginal and well within acceptable operational limits. Specifically, the proposed LDADM framework exhibits an average CPU usage of 3.1%, which is only 1-2% higher than CPU utilization of the state-of-the-art approaches. GASFI-IPP exhibits moderate CPU utilization, with average consumption of 2.7%. HGB-FP demonstrates slightly lower consumption, with an average CPU utilization of 2.4%. Finally, PeakSAX exhibits minimal utilization, with an average CPU consumption of 1%, respectively. The CPU utilization of the proposed framework remains below acceptable operational limits for the majority of SDN controller implementations, despite occasional surges during GAN activity. This additional overhead does not slow down performance or create bottlenecks in the data layer. The results indicate that the enhanced detection capabilities of the GAN can be achieved with minimal additional utilization of control plane resources. The small trade-off in computational overhead is justified by the significant performance gains achieved. This makes the proposed method a practical and efficient choice

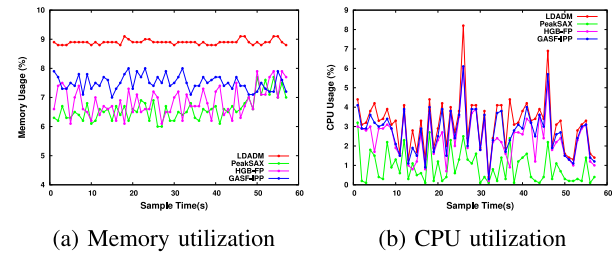


Fig. 15. Comparative analysis of computational resource utilization.

for real-world deployment where detection performance is critical.

The proposed LDADM framework for detecting and mitigating LDoS attacks is specifically designed for SDN environments. Its effectiveness relies heavily on the architectural principles of SDN. The absence of centralized control in non-SDN environment leads to fragmented and device-level data, which significantly reduces detection accuracy and increases response latency. The lack of programmable APIs in non-SDN devices prevents retrieving fine-grained flow-level statistics or deploying mitigation rules quickly. It must rely on indirect methods like polling or manual configuration, which is slower and inconsistent. The packet-based or interface-based monitoring in non-SDN environment requires extra hardware/software to reconstruct flows. This significantly increases processing overhead and delay of the LDoS attack detection. Moreover, in non-SDN networks, relying on offline methods limits the real-time responsiveness necessary to counter stealthy LDoS attacks. Each network device operates on its own, which makes it difficult to analyze data thoroughly and respond quickly, both of which are important for the effectiveness of the proposed framework. Hence, limitations of traditional non-SDN environments significantly limit the practicality and effectiveness of implementing the proposed approach.

VI. CONCLUSION

This study proposed a real-time GAN-based framework for the detection and mitigation of LDoS attacks. We utilized a GAN model to identify the LDoS attacks based on a neutral threshold. The LDADM also identifies the attack flows by examining sudden spikes from the source. The mitigation module in the LDADM protects SDN by continuously monitoring and identifying malicious traffic from attackers' IP addresses and enforcing mitigation flow rules on switches. Consequently, the mitigation module adds the attackers' IP addresses to the blocklist and halts the attack traffic for a specific duration through the controller, and executes flow regulations. Simulation results indicate that the LDADM achieves higher performance measures, enabling effective detection of LDoS attacks and quick mitigation response in an average of 8.58 seconds.

REFERENCES

- [1] Q. Li et al., "A comprehensive survey on DDoS defense systems: New trends and challenges," *Comput. Netw.*, vol. 233, Sep. 2023, Art. no. 109895.

- [2] M. Guirguis, A. Bestavros, and I. Matta, "Exploiting the transients of adaptation for RoQ attacks on Internet resources," in *Proc. 12th IEEE Int. Conf. Netw. Protoc.*, 2004, pp. 184–195.
- [3] M. U. Rahman, Z. U. Rahman, M. Fayaz, S. Abbas, and R. K. ShahSani, "Performance analysis of TCP/AQM under low-rate denial-of-service attacks," in *Proc. Int. Conf. Invent. Comput. Technol. (ICICT)*, vol. 3, 2016, pp. 1–5.
- [4] P. Lin, K. Ye, and C.-Z. Xu, "Dynamic network anomaly detection system by using deep learning techniques," in *Proc. Int. Conf. Cloud Comput.*, 2019, pp. 161–176.
- [5] J. Dromard and P. Owezarski, *Study and Evaluation of Unsupervised Algorithms Used in Network Anomaly Detection*, K. Arai, R. Bhatia, and S. Kapoor, Eds., San Francisco, CA, USA: Springer Int. Publ., 2020, pp. 397–416.
- [6] A. Smiiti, "A critical overview of outlier detection methods," *Comput. Sci. Rev.*, vol. 38, Nov. 2020, Art. no. 100306.
- [7] N. Huyan, X. Zhang, H. Zhou, and L. Jiao, "Hyperspectral anomaly detection via background and potential anomaly dictionaries construction," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 4, pp. 2263–2276, Apr. 2019.
- [8] A. Boukerche, L. Zheng, and O. Alfandi, "Outlier detection: Methods, models, and classification," *ACM Comput. Surv.*, vol. 53, pp. 1–37, Jun. 2020.
- [9] X. Xia et al., "Gan-based anomaly detection: A review," *Neurocomput.*, vol. 493, pp. 497–535, Jul. 2022.
- [10] S. Alnawayseh, W. Al-Sit, and T. Ghazal, "Smart congestion control in 5G/6G networks using hybrid deep learning techniques," *Complexity*, vol. 2022, pp. 1–10, Oct. 2022.
- [11] X. Luo and R. Chang, "On a new class of pulsing denial-of-service attacks and the defense," in *Proc. NDSS Symp.*, 2005, pp. 1–19.
- [12] J. Cao et al., "The crosspath attack: Disrupting the SDN control channel via shared links," in *Proc. 28th USENIX Secur. Symp.*, 2019, pp. 19–36.
- [13] H. Liu, "A new form of DoS attack in a cloud and its avoidance mechanism," in *Proc. ACM Workshop Cloud Comput. Secur. Workshop*, 2010, pp. 65–76.
- [14] D. Tang, X. Wang, Y. Yan, D. Zhang, and H. Zhao, "ADMS: An online attack detection and mitigation system for LDoS attacks via SDN," *Comput. Commun.*, vol. 181, pp. 454–471, Jan. 2022.
- [15] J. Wang and L. Wang, "SDN-defend: A lightweight online attack detection and mitigation system for DDoS attacks in SDN," *Sensors*, vol. 22, p. 8287, Oct. 2022.
- [16] J. E. Varghese and B. Muniyal, "An efficient IDS framework for DDoS attacks in SDN environment," *IEEE Access*, vol. 9, pp. 69680–69699, 2021.
- [17] Y. Ling et al., "Real-time detection of DDoS attacks based on hurst index," in *Proc. 2nd Int. Conf. Netw. Syst. AI (INSAI)*, 2022, pp. 42–45.
- [18] L. F. Eliyan and R. D. Pietro, "DeMi: A solution to detect and mitigate DoS attacks in SDN," *IEEE Access*, vol. 11, pp. 82477–82495, 2023.
- [19] V. Hnamte, A. A. Najjar, H. Nhung-Nguyen, J. Hussain, and M. N. Sugali, "DDoS attack detection and mitigation using deep neural network in SDN environment," *Comput. Secur.*, vol. 138, Mar. 2024, Art. no. 103661. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167404823005710>
- [20] H. Wang and Y. Li, "Overview of DDoS attack detection in software-defined networks," *IEEE Access*, vol. 12, pp. 38351–38381, 2024.
- [21] S. Yeom, C. Choi, and K. Kim, "LSTM-based collaborative source-side DDoS attack detection," *IEEE Access*, vol. 10, pp. 44033–44045, 2022.
- [22] D. Tang, C. Gao, X. Li, W. Liang, S. Xiao, and Q. Yang, "A detection and mitigation scheme of LDoS attacks via SDN based on the FSS-RSR algorithm," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 4, pp. 1952–1963, Jul./Aug. 2023.
- [23] D. Tang, S. Wang, B. Liu, W. Jin, and J. Zhang, "GASF-IPP: Detection and mitigation of LDoS attack in SDN," *IEEE Trans. Services Comput.*, vol. 16, no. 5, pp. 3373–3384, Sep./Oct. 2023.
- [24] D. Tang, X. Wang, X. Li, P. Vijayakumar, and N. Kumar, "AKN-FGD: Adaptive kohonen network based fine-grained detection of LDoS attacks," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 1, pp. 273–287, Jan./Feb. 2023.
- [25] D. Tang, Z. Zheng, X. Wang, S. Xiao, and Q. Yang, "PeakSAX: Real-time monitoring and mitigation system for LDoS attack in SDN," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 3, pp. 3686–3698, Sep. 2023.
- [26] D. Tang, S. Zhang, Y. Yan, J. Chen, and Z. Qin, "Real-time detection and mitigation of LDoS attacks in the SDN using the HGB-FP algorithm," *IEEE Trans. Services Comput.*, vol. 15, no. 6, pp. 3471–3484, Nov./Dec. 2022.
- [27] D. Tang, Y. Yan, S. Zhang, J. Chen, and Z. Qin, "Performance and features: Mitigating the low-rate TCP-targeted DoS attack via SDN," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 1, pp. 428–444, Jan. 2022.
- [28] Q. Wang, Y. Tian, L. Huang, J. Chen, C. Zheng, and Y. Lu, "SDN-based LDoS mitigation system," in *Proc. 2nd Int. Conf. Artif. Intell. Comput. Inf. Technol. (AICIT)*, 2023.
- [29] R. Xie, M. Xu, J. Cao, and Q. Li, "SoftGuard: Defend against the low-rate TCP attack in SDN," in *Proc. IEEE Int. Conf. Commun. (ICC)*, 2019, pp. 1–6.
- [30] B. Liu, D. Tang, J. Chen, W. Liang, Y. Liu, and Q. Yang, "ERT-EDR: Online defense framework for TCP-targeted LDoS attacks in SDN," *Expert Syst. Appl.*, vol. 254, Nov. 2024, Art. no. 124356.
- [31] M. Yue, Q. Yan, Z. Lu, and Z. Wu, "CCS: A cross-plane collaboration strategy to defend against LDoS attacks in SDN," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 3, pp. 3522–3536, Jun. 2024.
- [32] X. Li, N. Luo, D. Tang, Z. Zheng, Z. Qin, and X. Gao, "BA-BNN: Detect LDoS attacks in SDN based on bat algorithm and BP neural network," in *Proc. IEEE Int. Conf. Parallel Distrib. Process. Appl., Big Data Cloud Comput., Sustain. Comput. Commun., Soc. Comput. Netw. (ISPA/BDCloud/SocialCom/SustainCom)*, 2021, pp. 300–307.
- [33] J. Chen, Y. Tian, L. Huang, J. Jiao, Q. Wang, and C. Tang, "Detection of LDoS system based on periodic comparison and CNN," in *Proc. 2nd Int. Conf. Artif. Intell. Comput. Inf. Technol. (AICIT)*, 2023, pp. 1–4.
- [34] B. Liu, D. Tang, Y. Yan, Z. Zheng, S. Zhang, and J. Zhou, "TS-SVM: Detect LDoS attack in SDN based on two-step self-adjusting SVM," in *Proc. IEEE 20th Int. Conf. Trust, Security Privacy Comput. Commun. (TrustCom)*, 2021, pp. 678–685.
- [35] J. Deng, L. Cheng, H. Yuan, K. Zheng, X. Li, and Q. Li, "An online detection system for LDoS attack based on XGBoost," in *Proc. IEEE Int. Conf. Parallel Distrib. Process. Appl., Big Data Cloud Comput., Sustain. Comput. Commun., Soc. Comput. Netw. (ISPA/BDCloud/SocialCom/SustainCom)*, 2023, pp. 1083–1088.
- [36] Z. Liu, J. Yu, B. Yan, and G. Wang, "A deep 1-D CNN and bidirectional LSTM ensemble model with arbitration mechanism for LDDoS attack detection," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 6, no. 6, pp. 1396–1410, Dec. 2022.
- [37] C.-S. Shieh et al., "Detection of adversarial DDoS attacks using generative adversarial networks with dual discriminators," *Symmetry*, vol. 14, p. 66, Jan. 2022.
- [38] V. G. Da Silva Ruffo, L. F. Carvalho, J. Lloret, and M. L. Proenca, "Unsupervised DDoS detection using entropy features and F-AnoGAN in software-defined networks," in *Proc. 11th Int. Conf. Softw. Defined Syst. (SDS)*, 2024, pp. 35–41.
- [39] A. Mustapha et al., "Detecting DDoS attacks using adversarial neural network," *Comput. Secur.*, vol. 127, Apr. 2023, Art. no. 103117.
- [40] R. I. Nekvi, S. Saha, Y. A. Mtawa, and A. Haque, "Examining generative adversarial network for smart home DDoS traffic generation," in *Proc. Int. Symp. Netw., Comput. Commun. (ISNCC)*, 2023, pp. 1–6.
- [41] Liu, Luo, Jiang, Wang, Li, and Jia, "A CGAN-based DDoS attack detection method in SDN," in *Proc. Int. Wireless Commun. Mobile Comput. (IWCMC)*, 2021, pp. 1030–1034.
- [42] Liu, Luo, Jiang, Wang, Li, and Jia, "A CGAN-based DDoS attack detection method in SDN," in *Proc. Int. Wireless Commun. Mobile Comput. (IWCMC)*, 2021, pp. 1030–1034, doi: [10.1109/IWCMC51323.2021.9498840](https://doi.org/10.1109/IWCMC51323.2021.9498840).
- [43] M. Abdelaty, S. Scott-Hayward, R. Doriguzzi Corin, and D. Siracusa, "GADoT: Gan-based adversarial training for robust DDoS attack detection," in *Proc. IEEE Conf. Commun. Netw. Secur. (CNS)*, 2022, pp. 119–127.
- [44] C.-S. Shieh, T.-T. Nguyen, W.-W. Lin, M.-F. Horng, T.-V. Nguyen, and D. Miu, "Generating adversarial DDoS attacks with cyclegan architecture," in *Proc. Int. Conf. Comput. Technol. (ICCTech)*, 2022, pp. 64–69.
- [45] L. H. De Melo, G. D. C. Bertoli, M. Nogueira, A. L. Dos Santos, and L. A. Pereira, "Anomaly-flow: A multi-domain federated generative adversarial network for distributed denial-of-service detection," *IEEE Netw.*, early access, May 5, 2025, doi: [10.1109/MNET.2025.3567251](https://doi.org/10.1109/MNET.2025.3567251).
- [46] S. Saka, A. Al-Ataby, and V. Selis, "Generating synthetic tabular data for DDoS detection using generative models," in *Proc. IEEE 22nd Int. Conf. Trust, Security Privacy Comput. Commun. (TrustCom)*, 2023, pp. 1436–1442.
- [47] M. Komarchesqui, D. M. B. Lent, V. G. Da Silva Ruffo, L. F. Carvalho, J. Lloret, and M. L. Proenca, "Explainable AI feature selection in generative adversarial networks system aiming to detect DDoS attacks," in *Proc. 11th Int. Conf. Softw. Def. Syst. (SDS)*, 2024, pp. 27–34.

- [48] D. Lent, V. Ruffo, L. Carvalho, J. Lloret, J. Rodrigues, and M. Proença, "An unsupervised generative adversarial network system to detect DDoS attacks in SDN," *IEEE Access*, vol. 12, pp. 70690–70706, 2024.
- [49] R. Chauhan and S. Shah Heydari, "Polymorphic adversarial DDoS attack on IDS using GAN," in *Proc. Int. Symp. Netw., Comput. Communications (ISNCC)*, 2020, pp. 1–6.
- [50] "Mininet," 2022. [Online]. Available: <http://mininet.org/>
- [51] S. Knight, H. Nguyen, N. Falkner, R. Bowden, and M. Roughan, "The Internet topology zoo," *IEEE J. Sel. Areas Commun.*, vol. 29, no. 9, pp. 1765–1775, Oct. 2011.



Balaprakasa Rao Killi (Senior Member, IEEE) received the Ph.D. degree in computer science and engineering from the Indian Institute of Technology Guwahati. He is currently an Assistant Professor with the National Institute of Technology at Warangal, Warangal, India. His research interests include network function virtualization, software-defined networks, AI enabled networks, and network optimization.



Manjuluri Anil Kumar received the B.Tech. and M.Tech. degrees in computer science and engineering from Jawaharlal Nehru Technological University, Kakinada, India. He is currently pursuing the Ph.D. degree with the Department of Computer Science and Engineering, National Institute of Technology at Warangal, Warangal, India. His research interests include software-defined networks and AI enabled networks.



Eiji Oki (Fellow, IEEE) is a Professor with Kyoto University, Kyoto, Japan. He was with Nippon Telegraph and Telephone Corporation Laboratories, Tokyo, from 1993 to 2008, and the University of Electro-Communications, Tokyo, from 2008 to 2017. From 2000 to 2001, he was a Visiting Scholar with Polytechnic University, Brooklyn, NY, USA. His research interests include routing, switching, protocols, optimization, and traffic engineering in communication and information networks.